



Python antes do POO

O guia que você precisava antes de criar suas primeiras classes

Do zero ao POO — sem enrolação

Guia baseado no Curso em Vídeo — Prof. Gustavo Guanabara



O que você vai aprender

1



Condições com elif

.....

2



Repetição com for

.....

3



Repetição com while

.....

4



Tuplas

.....

5



Listas

.....

6



Dicionários

.....

7



Funções

.....

8



Módulos

.....

9



Tratamento de Erros (try/except)

.....



Condições com elif

Quando o if/else não é suficiente

Você já sabe usar `if` e `else`. Mas e quando você tem **mais de duas opções**?



Imagine um semáforo: ele não tem só verde e vermelho. Tem o amarelo também! O `elif` é exatamente isso — um caso do meio, entre o `if` e o `else`.

Como funciona

O Python testa as condições **de cima para baixo**. Quando encontra uma verdadeira, executa aquele bloco e pula os outros.

PYTHON – CONDIÇÕES COM ELIF

```
# Imagine que você quer classificar a nota de um aluno
nota = 7.5

if nota >= 9:
    print("Excelente! Nota A")
elif nota >= 7:
    print("Bom! Nota B")
elif nota >= 5:
    print("Regular. Nota C")
else:
    print("Reprovado. Nota D")
```

► SAÍDA

```
Bom! Nota B
```



Você pode ter **quantos** `elif` quiser! Mas só pode ter **um** `if` no início e **um** `else` no final (e o else é opcional).

Exemplo prático: classificando o IMC

PYTHON – ELIF NA PRÁTICA

```
imc = float(input("Qual o seu IMC? "))

if imc < 18.5:
    print("Abaixo do peso")
elif imc < 25:
    print("Peso normal – parabéns!")
elif imc < 30:
    print("Sobrepeso")
else:
    print("Obesidade")
```



Resumindo o elif

- `if` → testa a primeira condição
- `elif` → testa condições alternativas (pode repetir quantas vezes precisar)
- `else` → o que fazer se nenhuma condição for verdadeira
- Python para no **primeiro** bloco verdadeiro e ignora o resto



Repetição com for

Quando você sabe quantas vezes vai repetir

O `for` serve para **repetir um bloco de código** um número determinado de vezes, ou para **percorrer todos os itens** de uma sequência.



Imagine uma fila de frutas numa esteira: você pega cada fruta, uma por uma, e faz algo com ela. O `for` é exatamente isso — ele pega cada item de uma sequência e executa seu código para cada um.

Usando range() — contando números

PYTHON – FOR COM RANGE

```
# range(5) gera os números: 0, 1, 2, 3, 4
for i in range(5):
    print(f"Número: {i}")
```

► SAÍDA

```
Número: 0 Número: 1 Número: 2 Número: 3 Número: 4
```



`range(início, fim, passo)` — o **início** é incluído, o **fim** não. Por padrão começa em 0 e o passo é 1.

PYTHON – RANGE COM INÍCIO E FIM

```
# De 1 até 10, de 2 em 2 (só os ímpares)
for n in range(1, 11, 2):
    print(n, end=" ")
```

► SAÍDA

1 3 5 7 9

Percorrendo textos e listas

PYTHON – FOR PERCORRENDO UMA STRING

```
nome = "Python"
for letra in nome:
    print(letra)
```

► SAÍDA

```
P y t h o n
```

break e continue — controlando o loop

PYTHON – BREAK E CONTINUE

```
# break → sai do loop imediatamente
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")
# saída: 0 1 2 3 4

# continue → pula para a próxima repetição
for i in range(6):
    if i == 3:
        continue
    print(i, end=" ")
# saída: 0 1 2 4 5
```

Resumindo o for

- Use `for` quando você sabe **quantas vezes** vai repetir
- `range(n)` gera números de 0 até n-1
- `range(a, b)` gera de a até b-1
- `range(a, b, passo)` pula de passo em passo

- `break` para o loop; `continue` pula para a próxima volta



Repetição com while

Quando você repete enquanto uma condição for verdadeira

O `while` repete um bloco de código **enquanto uma condição for verdadeira**.

Diferente do `for`, você não precisa saber quantas vezes vai repetir.



Pense num videogame: o jogo continua rodando enquanto o jogador tiver vidas. Quando as vidas acabam, o loop termina. O `while` funciona igualzinho!

PYTHON – WHILE BÁSICO

```
contador = 1

while contador <= 5:
    print(f"Contando: {contador}")
    contador += 1  # IMPORTANTE: nunca esqueça de atualizar!

print("Acabou!")
```

► SAÍDA

```
Contando: 1 Contando: 2 Contando: 3 Contando: 4 Contando: 5 Acabou!
```



Se você esquecer de atualizar a variável dentro do `while`, ele vai rodar **para sempre** (loop infinito). Sempre verifique se a condição vai se tornar falsa em algum momento!

Uso clássico: validando entrada do usuário

PYTHON – WHILE PARA VALIDAR DADOS


```
# Fica pedindo a nota até o usuário digitar algo válido
nota = float(input("Digite uma nota de 0 a 10: "))

while nota < 0 or nota > 10:
    print("Nota inválida! Tente novamente.")
    nota = float(input("Digite uma nota de 0 a 10: "))

print(f"Nota registrada: {nota}")
```

For vs While — quando usar cada um?

Situação	Use
Sei quantas vezes vou repetir	for
Quero percorrer uma lista/tupla/string	for
Repito até o usuário fazer algo certo	while
Não sei quando a repetição vai parar	while

Resumindo o while

- `while condição:` — repete enquanto a condição for verdadeira
- Sempre garanta que a condição vai se tornar falsa em algum momento
- Muito útil para **validar entradas** do usuário
- Pode usar `break` para sair do loop quando quiser

CAPÍTULO 4



Tuplas

Uma coleção que ninguém pode mexer

Uma **tupla** é uma sequência de valores que **não pode ser alterada** depois de criada. Ela guarda vários dados em um único lugar.



Imagine uma pedra gravada: depois que você grava algo nela, não dá para apagar. Uma tupla é assim — depois de criada, os dados ficam fixos para sempre.

PYTHON – CRIANDO E USANDO TUPLAS

```
# Tupla com os dias úteis da semana
dias = ("Segunda", "Terça", "Quarta", "Quinta", "Sexta")

print(dias[0])      # Segunda (índice começa em 0!)
print(dias[-1])     # Sexta (índice negativo = do final)
print(len(dias))    # 5 (quantidade de itens)

# Percorrendo a tupla com for
for dia in dias:
    print(f"Dia: {dia}")
```

► SAÍDA

```
Segunda Sexta 5 Dia: Segunda Dia: Terça ... (e assim por diante)
```

Operações úteis com tuplas

PYTHON – OPERAÇÕES COM TUPLAS

```
numeros = (3, 7, 2, 9, 1)

print(min(numeros))    # 1 – menor valor
print(max(numeros))    # 9 – maior valor
```

```
print(sum(numeros))      # 22 - soma de tudo

# Verificando se um item está na tupla
print(7 in numeros)      # True
print(5 in numeros)      # False

# Contando ocorrências
notas = (7, 8, 7, 9, 7)
print(notas.count(7))    # 3 (o número 7 aparece 3 vezes)
```

Resumindo as tuplas

- Criada com **parênteses**: `(a, b, c)`
- **Imutável** — não dá para adicionar, remover ou alterar itens
- Índice começa em **0**; índice **negativo** conta do final
- Funções úteis: `len()`, `min()`, `max()`, `sum()`, `.count()`
- Use quando os dados **não devem mudar** (dias da semana, meses, coordenadas)

CAPÍTULO 5



Listas

Uma coleção que você pode mudar quando quiser

A **lista** é como uma tupla, mas você pode **adicionar, remover e alterar** itens livremente. É uma das estruturas mais usadas em Python.



Pense numa lista de compras: você escreve os itens, risca o que já comprou, adiciona novos produtos. Uma lista em Python é exatamente isso — flexível e editável.

PYTHON – CRIANDO E MODIFICANDO LISTAS

```
# Criando uma lista (colchetes!)
compras = ["arroz", "feijão", "macarrão"]

# Adicionando itens
compras.append("café")           # adiciona no final
compras.insert(1, "sal")         # adiciona na posição 1

# Removendo itens
compras.remove("macarrão")       # remove pelo valor
compras.pop()                   # remove o último
compras.pop(0)                  # remove o índice 0

# Alterando um item
compras[0] = "arroz integral"

print(compras)
```

Percorrendo e ordenando listas

PYTHON – FOR EM LISTAS E ORDENAÇÃO

```
notas = [7.5, 9.0, 6.0, 8.5, 5.0]
```

```
# Percorrendo
for nota in notas:
    print(f"Nota: {nota}")

# Ordenando (modifica a lista)
notas.sort()
print(notas)    # [5.0, 6.0, 7.5, 8.5, 9.0]

notas.sort(reverse=True)
print(notas)    # [9.0, 8.5, 7.5, 6.0, 5.0]

# Estatísticas
print(f"Maior: {max(notas)}")
print(f"Menor: {min(notas)}")
print(f"Média: {sum(notas) / len(notas):.1f}")
```

Tupla vs Lista — qual usar?

Característica	Tupla ()	Lista []
Pode alterar depois?	❌ Não	✅ Sim
Mais rápida?	✅ Sim	❌ Um pouco mais lenta
Quando usar	Dados fixos (meses, dias)	Dados que mudam (carrinho, notas)

Resumindo as listas

- Criada com **colchetes**: `[a, b, c]`
- **Mutável** — pode adicionar, remover e alterar itens
- `.append(x)` — adiciona x no final
- `.insert(i, x)` — adiciona x na posição i
- `.remove(x)` — remove o primeiro x encontrado
- `.pop()` — remove e retorna o último item
- `.sort()` — ordena a lista



Dicionários

Guardando dados com nome e valor

Um **dicionário** guarda pares de `chave: valor`. Em vez de acessar um item pelo índice (0, 1, 2...), você acessa pelo **nome** da chave.



Pense numa ficha cadastral: ela tem campos como "nome", "idade", "cidade". Cada campo tem um nome (chave) e um conteúdo (valor). Um dicionário Python funciona exatamente assim!

PYTHON – CRIANDO E USANDO DICIONÁRIOS

```
# Criando um dicionário (chave: valor)
pessoa = {
    "nome": "Willian",
    "idade": 30,
    "cidade": "São Paulo"
}

# Acessando valores
print(pessoa["nome"])      # Willian
print(pessoa["idade"])     # 30

# Adicionando / alterando
pessoa["email"] = "w@email.com" # nova chave
pessoa["idade"] = 31           # altera existente

# Removendo
del pessoa["cidade"]

print(pessoa)
```

```
{'nome': 'Willian', 'idade': 31, 'email': 'w@email.com'}
```

Percorrendo um dicionário

PYTHON – ITERANDO DICIONÁRIOS

```
aluno = {"nome": "Ana", "nota": 9.5, "turma": "A"}

# Percorrendo chaves e valores juntos
for chave, valor in aluno.items():
    print(f"{chave}: {valor}")

# Acessando com segurança (sem erro se chave não existir)
print(aluno.get("email", "não informado"))
```

► SAÍDA

```
nome: Ana nota: 9.5 turma: A não informado
```



Use `.get(chave, valor_padrão)` para acessar chaves que talvez não existam. Isso evita erros no programa!



Resumindo os dicionários

- Criado com **chaves**: `{"chave": valor}`
- Acessa valores pelo **nome da chave**, não por posição
- `dict["chave"] = valor` — adiciona ou altera
- `del dict["chave"]` — remove uma chave
- `.items()` — retorna pares chave/valor para iterar
- `.get(chave, padrão)` — acesso seguro sem risco de erro

Funções

O fundamento dos métodos em POO

Uma **função** é um bloco de código com nome próprio que você pode **chamar quando precisar**. Em POO, os métodos de uma classe são exatamente funções.



Imagine um controle remoto: você aperta o botão "ligar" e a TV liga — sem precisar saber como funciona por dentro. Uma função é o mesmo conceito: você dá um nome para um bloco de código e chama ele quando precisar.

Criando e chamando funções

PYTHON – FUNÇÃO SIMPLES

```
# Definindo a função
def saudacao():
    print("Olá! Bem-vindo ao Python!")

# Chamando a função
saudacao()
saudacao() # pode chamar quantas vezes quiser
```

Funções com parâmetros

PYTHON – PARÂMETROS E RETURN

```
# Parâmetros são as "entradas" da função
def somar(a, b):
    resultado = a + b
    return resultado # return devolve o valor

total = somar(5, 3)
print(f"5 + 3 = {total}") # 5 + 3 = 8
```



```
# Parâmetros com valor padrão
def apresentar(nome, cargo="visitante"):
    print(f"Olá, {nome}! Você é {cargo}.")

apresentar("Willian", "desenvolvedor")
apresentar("João")    # usa o padrão "visitante"
```

► SAÍDA

```
Olá, Willian! Você é desenvolvedor. Olá, João! Você é visitante.
```

A conexão com POO

PYTHON – FUNÇÃO VS MÉTODO DE CLASSE

```
# Função normal
def calcular_area(largura, altura):
    return largura * altura

# O mesmo como método de uma classe (POO)
class Retangulo:
    def __init__(self, largura, altura):
        self.largura = largura
        self.altura = altura

    def calcular_area(self):    # self = a função conhece a classe!
        return self.largura * self.altura
```



Percebeu? Um **método de classe** é uma função com `self` como primeiro parâmetro. O `self` dá acesso aos atributos do objeto. Se você entende funções, já entende métodos!



Resumindo as funções

- `def nome_da_funcao(parâmetros):` define a função
- `return valor` devolve um resultado (opcional)
- Parâmetros com `=` têm valor padrão e são opcionais na chamada
- Funções evitam repetição de código — escreva uma vez, use várias

- Métodos de classe são funções com `self` como primeiro parâmetro



Módulos

Usando códigos prontos e organizando os seus

Um **módulo** é um arquivo Python com funções e variáveis prontas para usar. Em vez de escrever tudo do zero, você **importa** o que precisa.



Pense numa caixa de ferramentas: você não fabrica seus próprios martelos e chaves de fenda — você os pega da caixa. Módulos são essas caixas de ferramentas prontas!

Módulos mais úteis para o dia a dia

PYTHON – MÓDULO MATH

```
import math

print(math.sqrt(16))      # 4.0  – raiz quadrada
print(math.pi)           # 3.14159... – número pi
print(math.ceil(4.2))    # 5    – arredonda para cima
print(math.floor(4.8))   # 4    – arredonda para baixo
```

PYTHON – MÓDULO RANDOM

```
import random

# Número aleatório entre 1 e 10
numero = random.randint(1, 10)
print(f"Sorteio: {numero}")

# Escolher aleatoriamente de uma lista
frutas = ["maçã", "banana", "uva"]
escolha = random.choice(frutas)
print(f"Fruta sorteada: {escolha}")
```

```

from datetime import date, datetime

hoje = date.today()
print(f"Hoje: {hoje}")           # 2026-05-19
print(f"Ano: {hoje.year}")       # 2026
print(f"Mês: {hoje.month}")      # 5

agora = datetime.now()
print(f"Agora: {agora.strftime('%H:%M')}")

```

import vs from...import

Forma	Como usar	Quando usar
<code>import math</code>	<code>math.sqrt()</code>	Quando usa várias coisas do módulo
<code>from math import sqrt</code>	<code>sqrt()</code>	Quando usa só uma coisa específica
<code>import math as m</code>	<code>m.sqrt()</code>	Para encurtar nomes longos

Resumindo os módulos

- `import modulo` — importa o módulo completo
- `from modulo import funcao` — importa só o que precisa
- Módulos úteis: `math`, `random`, `datetime`, `os`
- Você também pode criar seus próprios módulos (arquivos .py)



Tratamento de Erros

Como fazer seu programa não travar quando algo dá errado

Programas reais sempre encontram situações inesperadas: o usuário digita uma letra quando deveria ser número, um arquivo não existe, a conexão cai. O **try/except** cuida desses casos sem deixar o programa travar.



Imagine um paraquedas: você salta do avião esperando o paraquedas funcionar (try), mas se falhar, você tem o paraquedas reserva (except). Seu programa nunca vai ao chão!

Estrutura básica

PYTHON – TRY/EXCEPT BÁSICO

```
try:
    # Código que pode dar erro
    numero = int(input("Digite um número: "))
    resultado = 100 / numero
    print(f"100 / {numero} = {resultado}")

except ValueError:
    # Usuário digitou uma letra
    print("Erro: você precisava digitar um número!")

except ZeroDivisionError:
    # Usuário digitou zero
    print("Erro: não é possível dividir por zero!")
```

finally — código que sempre executa

PYTHON – TRY/EXCEPT/ELSE/FINALLY

```

try:
    idade = int(input("Sua idade: "))

except ValueError:
    print("Isso não é uma idade válida!")

else:
    # Só executa se NÃO houve erro
    print(f"Você tem {idade} anos.")

finally:
    # Executa SEMPRE, com ou sem erro
    print("Programa finalizado.")

```

Erros mais comuns em Python

Erro	Quando acontece	Exemplo
<code>ValueError</code>	Tipo errado de dado	<code>int("abc")</code>
<code>ZeroDivisionError</code>	Divisão por zero	<code>10 / 0</code>
<code>TypeError</code>	Operação inválida entre tipos	<code>"a" + 1</code>
<code>IndexError</code>	Índice fora do range	<code>lista[99]</code> (lista tem 3 itens)
<code>KeyError</code>	Chave não existe no dicionário	<code>dict["chave_errada"]</code>
<code>FileNotFoundError</code>	Arquivo não encontrado	<code>open("nao_existe.txt")</code>

Aplicando no ContaBancaria (seu exercício!)

PYTHON – TRY/EXCEPT NA CLASSE CONTABANCARIA

```

class ContaBancaria:
    def __init__(self, id, nome, saldo=0):
        self.id = id
        self.titular = nome
        self.saldo = saldo

```

```
def sacar(self, valor):
    try:
        if valor <= 0:
            raise ValueError("Valor deve ser positivo")
        if valor > self.saldo:
            raise ValueError("Saldo insuficiente")
        self.saldo -= valor
        print(f"Saque de R${valor:.2f} realizado.")
    except ValueError as e:
        print(f"Erro no saque: {e}")
```

Resumindo o try/except

- `try:` — tenta executar o código
- `except TipoDeErro:` — captura erros específicos
- `else:` — executa só se não houve erro
- `finally:` — executa sempre (com ou sem erro)
- `raise` — você mesmo pode "lançar" um erro quando quiser
- Nunca use `except:` vazio — sempre especifique o tipo de erro



Agora você está pronto para o POO!

Com esses conceitos dominados, suas classes vão fazer muito mais sentido. Bons estudos!

Checklist final

- ✓ Condições com `elif`
- ✓ Repetição com `for`
- ✓ Repetição com `while`
- ✓ Tuplas `()`
- ✓ Listas `[]`
- ✓ Dicionários `{}`
- ✓ Funções e `return`
- ✓ Módulos e `import`
- ✓ `try/except` para erros